MAZE SOLVER

WITH KEYS AND DOORS

Why this design is a strong solution: algorithm, correctness, complexity, interface, and tradeoffs

Core question
How can we find a shortest route when what we can do depends on the keys collected so far?

Project implementation report

This report explains the choices in the Python/Tkinter project in maze-keys-and-doors. It focuses on why breadth-first search (BFS) is appropriate, how the key inventory changes the search, why the generator works, and what tradeoffs remain.

“Best” here means a solution that is correct for the requirements, returns a shortest path, stays understandable for an academic demonstration, and uses standard Python tools. It does not mean BFS has the smallest possible runtime on every individual maze.

Python 3 | Tkinter | BFS | CSV | Random generation

1. The problem and what makes it tricky

A maze is a square grid. A move goes one cell up, down, left, or right. Walls block movement. Keys are collected automatically and remain in the player inventory. A door can be crossed only after collecting its matching key. The requested output is the shortest valid start-to-exit route, or -1 if no route exists.

Without doors, each location is a graph vertex and each legal step is an edge. Keys make the graph depend on history: being at (row 2, column 3) with key a is a different situation from being at the same cell without key a. A search that marks only cells visited can incorrectly discard the useful second visit.

The project represents this full search state as (row, column, key mask). The mask is a compact set of bits: bit 0 means key a, bit 1 means key b, and so on. Up to six keys fit in one small integer.

2. Why BFS is the right shortest-path algorithm

Every allowed move costs exactly one step. There are no terrain costs, time penalties, or weighted edges. BFS explores all states at distance 0, then all states at distance 1, then distance 2, and continues in that order. Therefore, the first exit state removed from the queue has the fewest moves among all valid routes.

This gives two useful properties for the assignment: the solution is provably shortest, and the algorithm is easy to explain as a queue-based layer-by-layer search. Dijkstra’s algorithm would also find the answer but adds machinery for weights that this problem does not have. Depth-first search may find a route, but its first route is not guaranteed to be shortest.

State field	Meaning	Why it matters
(r, c)	Current row and column	Determines which adjacent cells can be considered.
mask	Keys collected so far	Determines which doors are traversable.
parent[state]	Previous state in the BFS tree	Reconstructs the actual route after reaching the exit.

BFS in plain language

```
queue = [start with empty inventory]
visited = {start state}
while queue is not empty:
    state = remove the oldest queued state
    if current cell is the exit: stop
    for each of the four neighboring cells:
        skip if outside the grid or a wall
        skip a door if its key bit is not set
        set a key bit when entering a key
        if the resulting (row, column, mask) is new:
            remember its parent and add it to the queue
```

The parent map stores one predecessor per discovered state. Once BFS finds the exit, following parent links backward reaches the start; reversing that list gives the route. The route contains both S and E, so number of moves equals number of route cells minus one.

3. A concrete key-and-door example

Consider this 4 by 4 maze. The player begins at S, collects key a, passes door A, and reaches E. The lower rows are isolated by walls.

S	a	A	E
#	#	#	#
.	.	.	.
.	.	.	.

State progression: Start at (0,0,000000). Move right to a and turn on its bit. Move right into A, which is allowed because a is in the mask. Move right into E. This route has 3 moves, which is the shortest possible because the exit is three cells to the right.

Why key-aware visited states are essential

Suppose a route reaches a junction without key a and cannot pass door A. It may later loop around, collect a, and return to that same junction. A cell-only visited set would say “this junction was already visited” and might suppress the useful state. The correct states are (junction, mask without a) and (junction, mask with a). They are distinct, so BFS explores both. This is the central correctness detail in the implementation.

Correctness argument

Soundness: BFS only adds legal neighbors. It rejects walls, bounds violations, and locked doors; keys update the mask as soon as they are entered. Every reconstructed path therefore follows the game rules.
Completeness: Every reachable state is eventually queued unless it has already been reached with the identical position and inventory, in which case exploring it again cannot reveal new options. **Shortest path:** Unit-cost moves and FIFO order ensure states are processed in nondecreasing move count. Thus the first exit state found has minimum distance.

4. Complexity and what “best case” means

For an N by N grid and T distinct key types, each cell can appear with at most 2^T possible inventories. The search graph therefore contains at most N² x 2^T states. Every state checks at most four neighbors. The parent map and queue store discovered states.

Measure	Bound	Reason
Time	O(N² x 2^T)	At most four neighbor checks per state; four is constant.
Space	O(N² x 2^T)	Visited and parent state records, plus queue.
At T = 6	At most 64 inventory masks per cell	The limit keeps key-state growth bounded and practical.

Best-case runtime: If the exit is directly adjacent and legal, BFS can stop after examining only a tiny part of the state graph. Worst-case runtime: If the exit is unreachable or far away, BFS may explore nearly every reachable position-inventory state. The project is a strong fit because BFS is both optimal for the stated move cost and simple to verify. No algorithm can promise a tiny search on every maze; in the worst case, the key inventory creates up to 64 variants per location.

5. Why the generator and GUI are designed this way

Random generation builds a route with a randomized self-avoiding walk, then puts walls and open cells across the grid while keeping the route clear. Keys appear before their matching doors on that known route. The generator then runs the same BFS solver before returning the maze. This validation step avoids accepting a maze that has no valid solution.

Keys and doors are now placed at spaced indexes along the carved route: keys occupy the earlier portion, and matching doors appear farther along. This makes them easier to distinguish in a visual demonstration than adjacent key-door pairs. The rest of the grid remains randomized, so extra open cells can create shortcuts. Consequently, a shortest valid solution may bypass some placed doors; the generator guarantees a solvable route, not that every door lies on the shortest route.

This last point is an intentional behavior. A variant that required every generated shortest path to pass every door would need to prevent shortcuts, usually by creating a single narrow corridor. That gives stronger control but less varied mazes. The current generator prioritizes a randomized-looking grid and a guaranteed solution.

The Tkinter renderer keeps maze data separate from animation state. The grid is not modified when a key is collected or a door is opened; the visualizer tracks collected/opened items separately. Tkinter after() schedules each movement so the interface can still respond to pause, resume, reset, and speed changes.

CSV export writes two files: the original maze, unchanged, and a same-sized path overlay containing 1 for route cells and 0 elsewhere. This preserves the identity of original keys and doors, which a single-character x marker would overwrite.

6. Architecture and demonstration flow

The project separates the algorithm from the interface so BFS can be checked without opening a GUI.

File	Responsibility
maze.py	Grid model and symbol/key-door validation.
solver.py	BFS over (row, column, key mask); returns path or no path.
generator.py	Solvable and intentionally unsolvable random maze generation.
csv_handler.py	CSV loading and lossless two-file solution export.
visualizer.py	Tkinter grid, status display, movement and key/door animation.
main.py	Desktop application entry point.
tests/test_maze.py	Unit tests for shortest paths, keys, validation, generation, and export.

Recommended demo: generate a maze with T = 3, show the separated key and door placement, solve it, then animate and pause/resume. Next check the unsolvable-maze option, generate another maze, and show the -1/no-route status. Finally load samples/solvable.csv or samples/unsolvable.csv and export one solution to demonstrate CSV handling.

Run commands from the maze-keys-and-doors folder:

```
python3 main.py
python3 -m unittest discover -s tests -v
```

7. Limitations and design decisions

Exit symbol overlap: The requested symbol set uses E for both the exit and the door for key e. A single-character cell cannot mean both. This project reserves E for the exit and uses G as the door symbol corresponding to key e. The README documents this extension.

Random routes may have shortcuts: As described above, keys and doors are spatially spread, but optional open cells can create shorter alternate routes that bypass doors. If a grading rubric explicitly requires every door to be traversed in each generated solution, the generator should be configured as a forced corridor rather than a shortcut-rich random grid.

Input validation: The loader checks square dimensions, supported symbols, one start, one exit, matching key/door types, and the supported key count. Invalid input is reported before solving.

Platform requirements: The app uses Python 3 and Tkinter, with no third-party Python packages. On Windows and macOS, use a Python distribution that includes Tkinter.

Conclusion

BFS on position-plus-inventory states is the best algorithmic fit because each move has equal cost and door access depends on collected keys. The state model preserves necessary revisits, BFS guarantees a shortest valid path, and parent links make the route easy to display. The modular design keeps the solver testable, while the GUI, generator, and two-file CSV export make the algorithm demonstrable as a complete student project.

Project files discussed: maze.py, solver.py, generator.py, csv_handler.py, visualizer.py, main.py, README.md, and tests/test_maze.py.